Memorando

Aplicações Móveis, 2026/2027

De: Emanuel Carneiro dos Santos
Nº de Matrícula: 20230429
Assunto: Componentes da Aplicação Android
Data: 10/11/2026

1. Introdução

O presente memorando descreve as experiências realizadas no **Laboratório 3 - Parte 1**, dedicado aos **componentes da aplicação Android** e aos fundamentos do seu ciclo de vida: o ciclo de vida da actividade, a pilha de retorno (back stack) e as tarefas. As aplicações foram desenvolvidas em **Java**, com a interface definida em layouts **XML**.

Uma aplicação Android é composta por quatro tipos de componentes, declarados no `AndroidManifest.xml` e activados pelo sistema operativo, e não directamente pelo programador. A **actividade** (activity) representa um ecrã com interface e atravessa estados notificados pelos callbacks `onCreate`, `onStart`, `onResume`, `onPause`, `onStop`, `onRestart` e `onDestroy`. O **serviço** (service) executa trabalho sem interface, independentemente da actividade que o iniciou. O **receptor de difusão** (broadcast receiver) reage a mensagens difundidas pelo sistema, seleccionadas por um intent-filter. O **fornecedor de conteúdo** (content provider) expõe dados a outras aplicações. Os componentes comunicam por Intents e podem partilhar estado através do contexto da aplicação (`Application`).

São quatro os exercícios abordados. O **Exercício I** estuda o ciclo de vida da actividade numa aplicação criada de raiz e na aplicação fornecida ActivityLifecycle, com quatro actividades. O **Exercício II** analisa o ciclo de vida de um serviço iniciado e a partilha de estado pelo contexto da aplicação. O **Exercício III** implementa um broadcast receiver que regista as mudanças de estado do telefone, com pedido de permissões em tempo de execução. O **Exercício IV**, apresentado como desafio na secção 3, consiste numa aplicação de bloco de notas com três actividades.

Quanto às restrições, usou-se o Android Studio Rabbit 2 | 2026.2.2 e um dispositivo Samsung SM-A105G (Android 11, API 30). Os projectos fornecidos exigiram versões anteriores do JDK no Gradle e a correcção da raiz dos seus layouts. No Exercício III, o nome e o número da chamada real foram parcialmente ocultados.

OBS: As respostas às perguntas do laboratório encontram-se no final de cada subsecção que contém perguntas.

2. Experiências Realizadas

Descrevem-se, de seguida, os Exercícios I a III do enunciado, pela ordem em que foram realizados. O Exercício IV é apresentado na secção 3 (3. Desafios).

2.1. Exercício I - Actividades

O objectivo deste exercício é compreender o ciclo de vida das actividades Android, observando a ordem em que o sistema invoca os callbacks em resposta às acções do utilizador.

2.1.1. Aplicação Simple Activity

O projecto foi criado com o modelo «Empty Views Activity» (Figura 1), que gera uma actividade em Java com layout XML. Na configuração (Figura 2) definiram-se o nome Simple Activity, o pacote `ao.co.isptec.aplm.simpleactivity`, a linguagem Java e o SDK mínimo API 24.

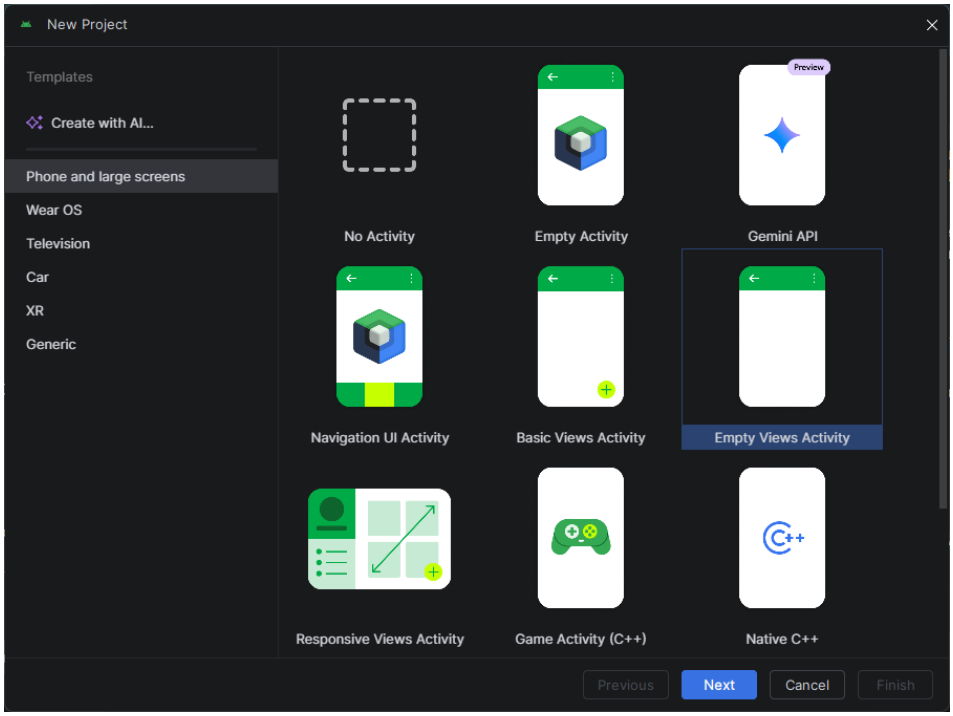


Figura 1 **–** Selecção do modelo Empty Views Activity, que gera uma actividade Java com layout XML

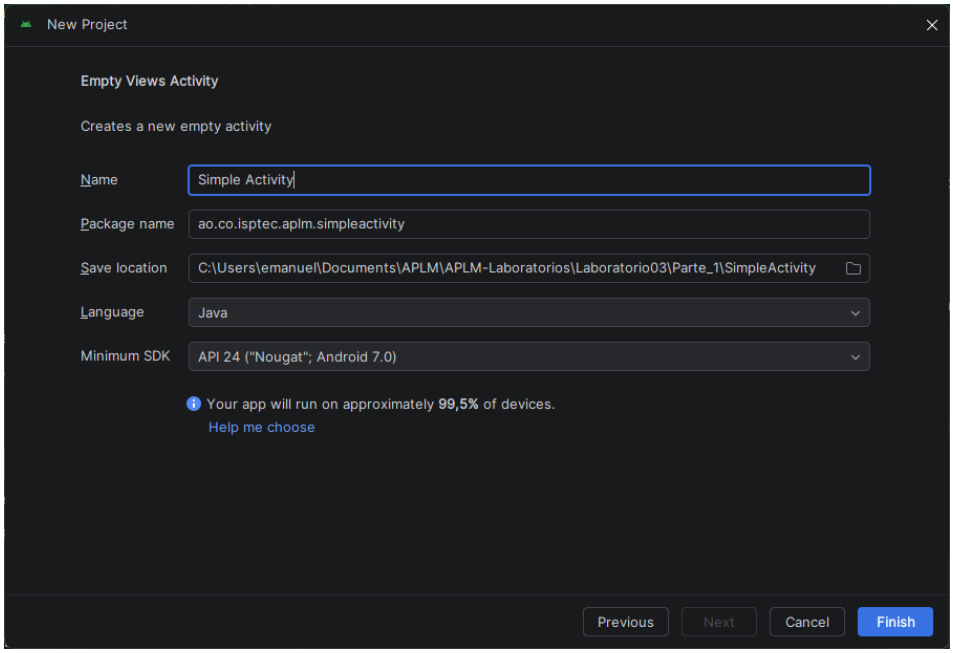


Figura 2 **–** Configuração do projecto Simple Activity: pacote `ao.co.isptec.aplm.simpleactivity`, linguagem Java e SDK mínimo API 24

Ao `activity_main.xml` acrescentou-se o botão «Encerrar», ancorado ao fundo do ecrã e ligado ao método `finishButtonPressed` pelo atributo `android:onClick`; o texto foi extraído para o recurso `@string/encerrar`:

```
<Button
    android:id="@+id/buttonEncerrar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginBottom="48dp"
    android:onClick="finishButtonPressed"
    android:text="@string/encerrar"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent" />
```

Na `MainActivity`, o método público recebe a `View` que originou o clique e termina a actividade com `finish()`:

```
public void finishButtonPressed(View view) {
    finish();
}
```

A Figura 3 mostra o código e a aplicação em execução no painel Running Devices; a Figura 4 confirma que o botão encerra a aplicação e devolve o utilizador ao ecrã inicial.

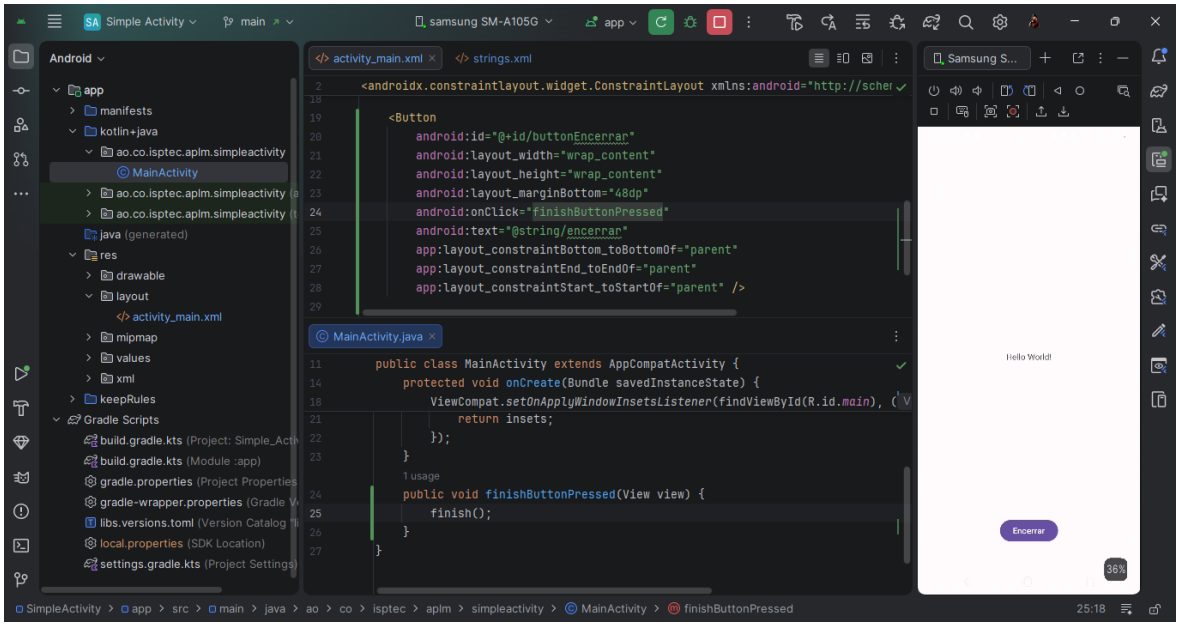


Figura 3 **–** Botão «Encerrar» no `activity_main.xml` e método `finishButtonPressed()` na `MainActivity`, com a aplicação espelhada no painel Running Devices



(a) (b)

Figura 4 **–** Execução no Samsung SM-A105G: (a) actividade principal com o botão «Encerrar»; (b) ecrã inicial do Android após o clique

2.1.2. Ciclo de vida da actividade

Para registar cada transição, sobrepuseram-se os seis callbacks do ciclo de vida e acrescentou-se uma mensagem de log ao `onCreate`. Cada método chama primeiro a implementação da superclasse, obrigatória em todos os callbacks, e regista o seu nome com o nível I e a tag `MainActivity`:

```
private static final String TAG = "MainActivity";

@Override protected void onStart()   { super.onStart();   Log.i(TAG, "onStart"); }
@Override protected void onResume()  { super.onResume();  Log.i(TAG, "onResume"); }
@Override protected void onPause()   { super.onPause();   Log.i(TAG, "onPause"); }
@Override protected void onStop()    { super.onStop();    Log.i(TAG, "onStop"); }
@Override protected void onRestart() { super.onRestart(); Log.i(TAG, "onRestart"); }
@Override protected void onDestroy() { super.onDestroy(); Log.i(TAG, "onDestroy"); }
```

As sequências foram observadas no Logcat com o filtro `package:mine tag:MainActivity`, que inclui também as mensagens do `ViewRootImpl` associadas à janela da actividade (Figuras 5 a 8).

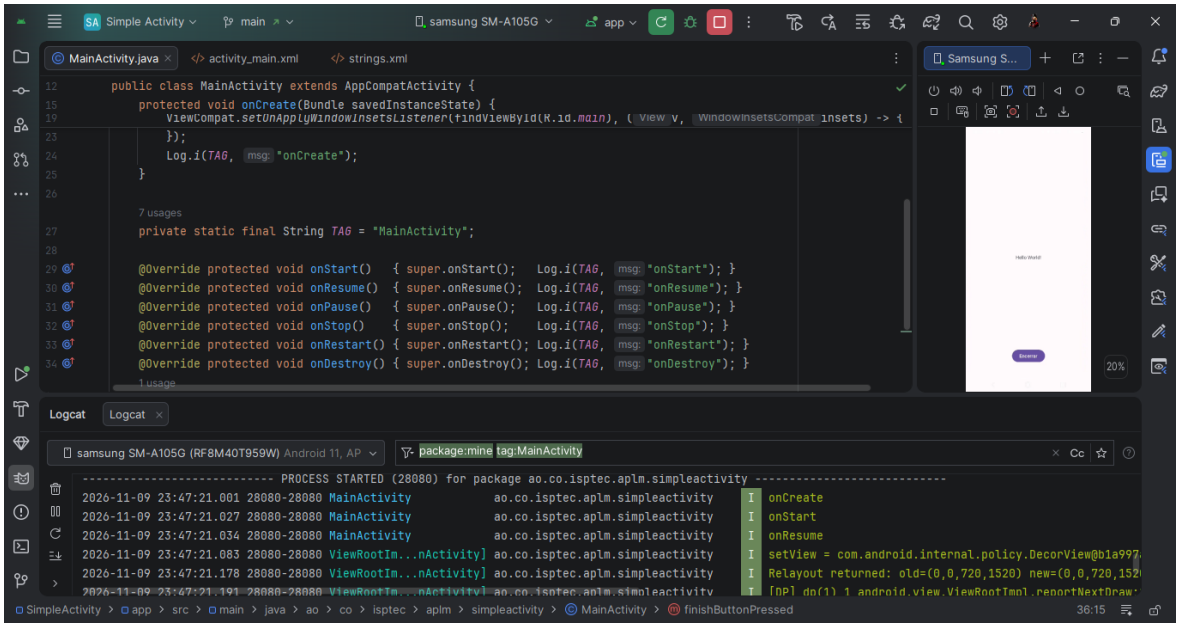


Figura 5 – Logcat após o deploy da aplicação: sequência `onCreate` → `onStart` → `onResume`

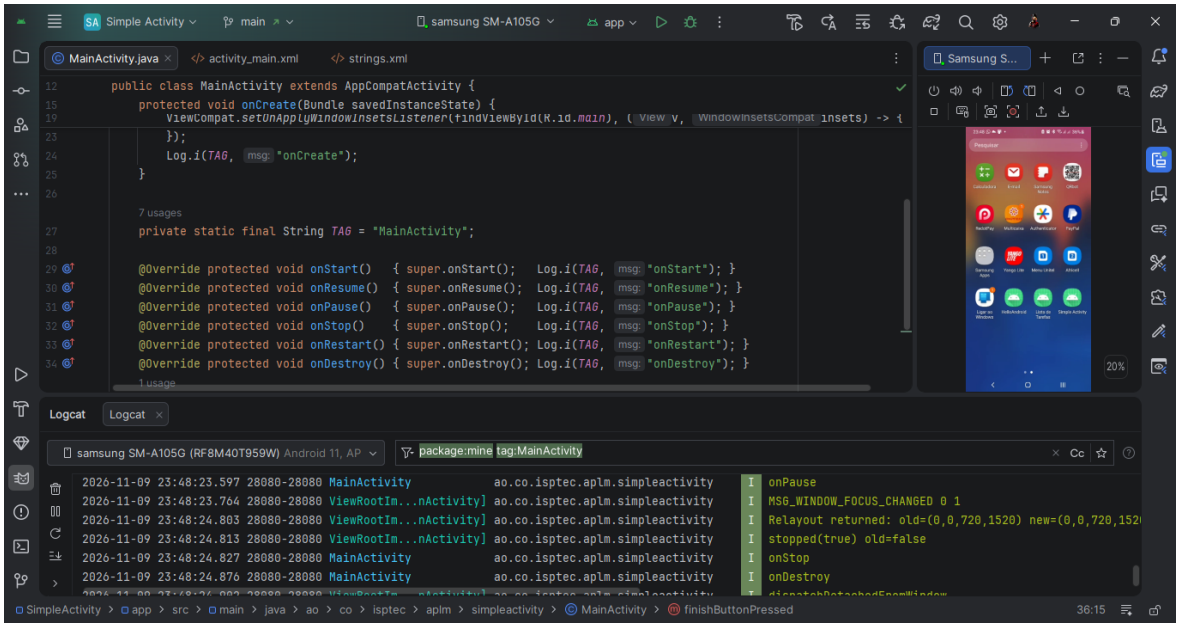


Figura 6 – Logcat após pressionar «Encerrar»: sequência `onPause` → `onStop` → `onDestroy`

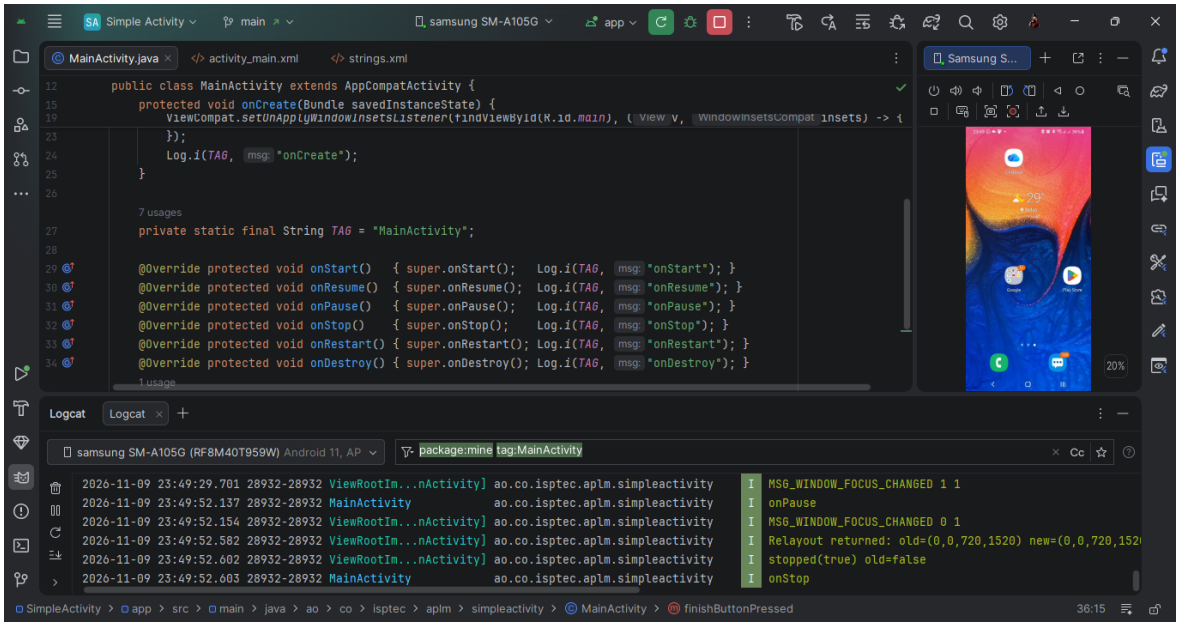


Figura 7 **–** Logcat após pressionar o botão Home: sequência `onPause` → `onStop`, sem `onDestroy`

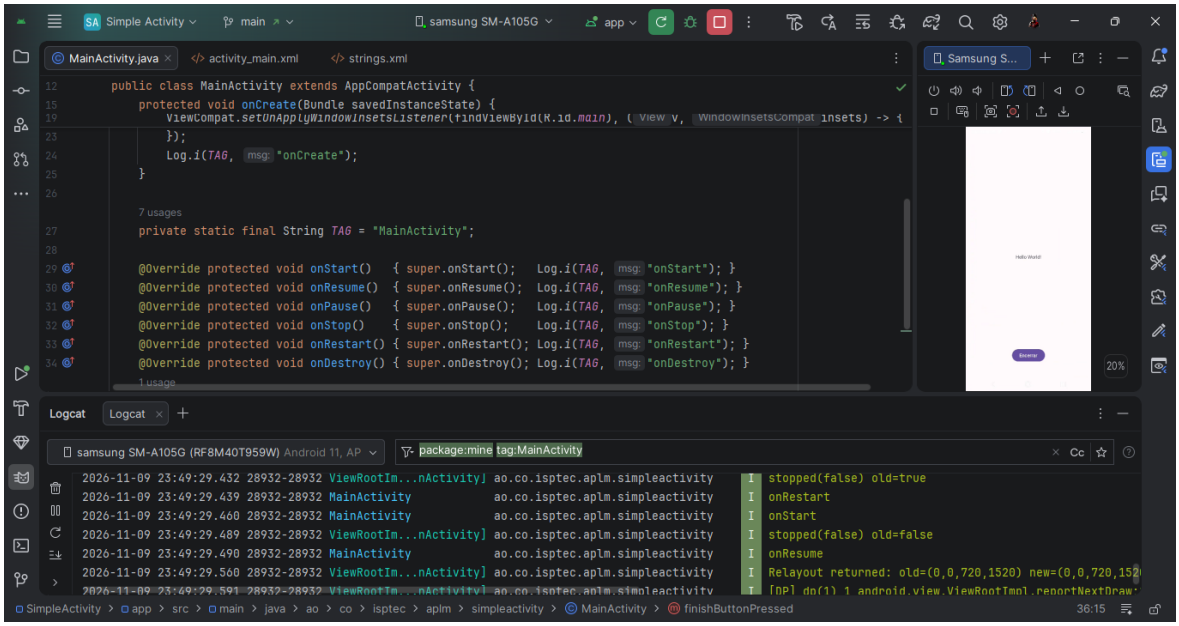


Figura 8 **–** Logcat ao retomar a aplicação a partir dos Recentes: sequência `onRestart` → `onStart` → `onResume`

a) Que sequência de mensagens é apresentada após o deploy da aplicação?

R:

`onCreate` → `onStart` → `onResume` (Figura 5). O sistema cria a actividade e carrega o layout (`onCreate`), torna-a visível (`onStart`) e coloca-a em primeiro plano, pronta a receber a interacção do utilizador (`onResume`).

b) Que sequência de estados ocorre ao pressionar o botão «Encerrar» e como se interpreta?

R:

`onPause` → `onStop` → `onDestroy` (Figura 6). A chamada a `finish()` retira a actividade da pilha de retorno: perde o primeiro plano (`onPause`), deixa de ser visível (`onStop`) e é destruída

(`onDestroy`), libertando os seus recursos. Sendo a única actividade da tarefa, a aplicação termina.

c) O que acontece ao pressionar o botão Home do dispositivo?

R:

Apenas `onPause` → `onStop` (Figura 7). A actividade passa para segundo plano mas não é destruída: mantém-se na pilha de retorno, com o seu estado em memória, até ser retomada ou até o sistema precisar de recursos.

d) Que sequência de mensagens é apresentada ao retomar a aplicação e porquê?

R:

`onRestart` → `onStart` → `onResume` (Figura 8). Como a instância ainda existe, o sistema não chama `onCreate`: `onRestart` assinala o regresso a partir do estado parado, seguindo-se a visibilidade e o primeiro plano. Abrir o ecrã de Recentes já provoca, por si só, `onPause` e `onStop`, porque a actividade deixa de estar em primeiro plano.

e) Que efeito tem a apresentação de um `Toast` no método `onResume`?

R:

Acrescentou-se ao `onResume` a instrução do enunciado:

```
@Override
protected void onResume() {
    super.onResume();
    Log.i(TAG, "onResume");
    Toast.makeText(this, "onResume", Toast.LENGTH_SHORT).show();
}
```

O Toast «onResume» surge sempre que a actividade volta ao primeiro plano: no arranque, no regresso do ecrã inicial ou dos Recentes e após o desbloqueio do ecrã (Figuras 9 e 10). Ao contrário do log, torna a transição visível no próprio dispositivo, sem bloquear a interacção.

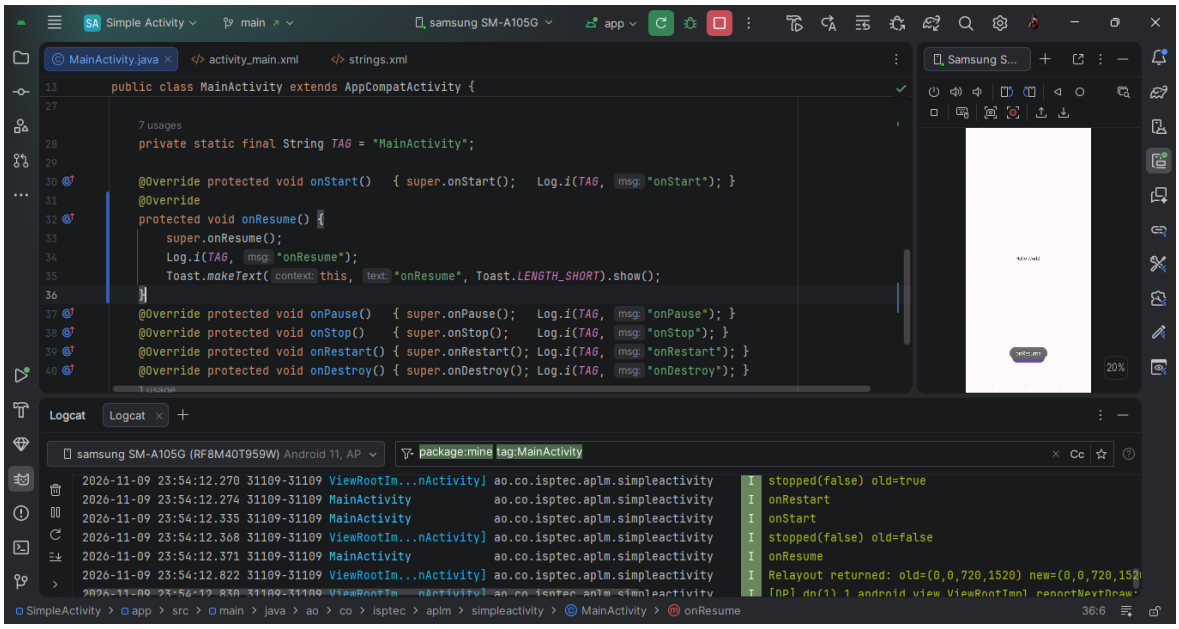


Figura 9 – Instrução `Toast.makeText()` no `onResume` e sequência `onRestart` → `onStart` → `onResume` no Logcat

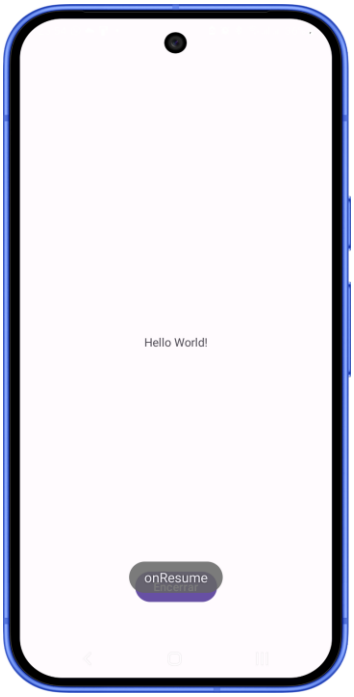


Figura 10 **–** Toast «onResume» apresentado no dispositivo ao retomar a actividade

2.1.3. Aplicação ActivityLifecycle

O projecto ActivityLifecycle, fornecido em arquivo, foi importado com **File** → **Open** e sincronizado com o Gradle JDK 11. É composto por três actividades principais (`ActivityA`, `ActivityB` e `ActivityC`), uma actividade de diálogo (`DialogActivity`) e duas classes auxiliares no pacote `util` (Figura 11): `StatusTracker`, um singleton que guarda a lista de callbacks invocados e o último estado de cada actividade, e `Utils`, que imprime essa informação no ecrã.

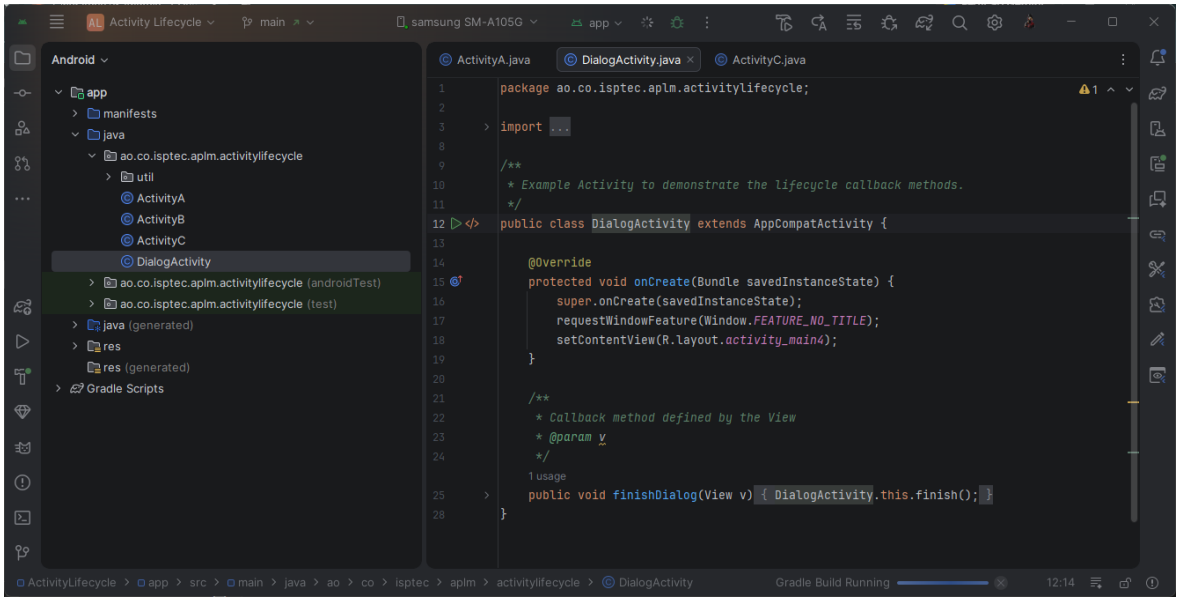


Figura 11 **–** Estrutura do projecto ActivityLifecycle importado: actividades A, B, C e `DialogActivity`, e classes auxiliares do pacote `util`

Na primeira execução, a interface apresentou-se sobreposta no canto superior (Figura 12). Os quatro layouts tinham como raiz um `ConstraintLayout` sem restrições (constraints), que

posiciona todas as vistas na origem; o atributo `android:orientation="vertical"` já presente revela que a intenção original era um `LinearLayout`. A substituição da raiz nos quatro ficheiros repôs a disposição vertical (Figura 13):

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    ...>
    <!-- título, botões, Lifecycle Method List e Activity Status -->
</LinearLayout>
```

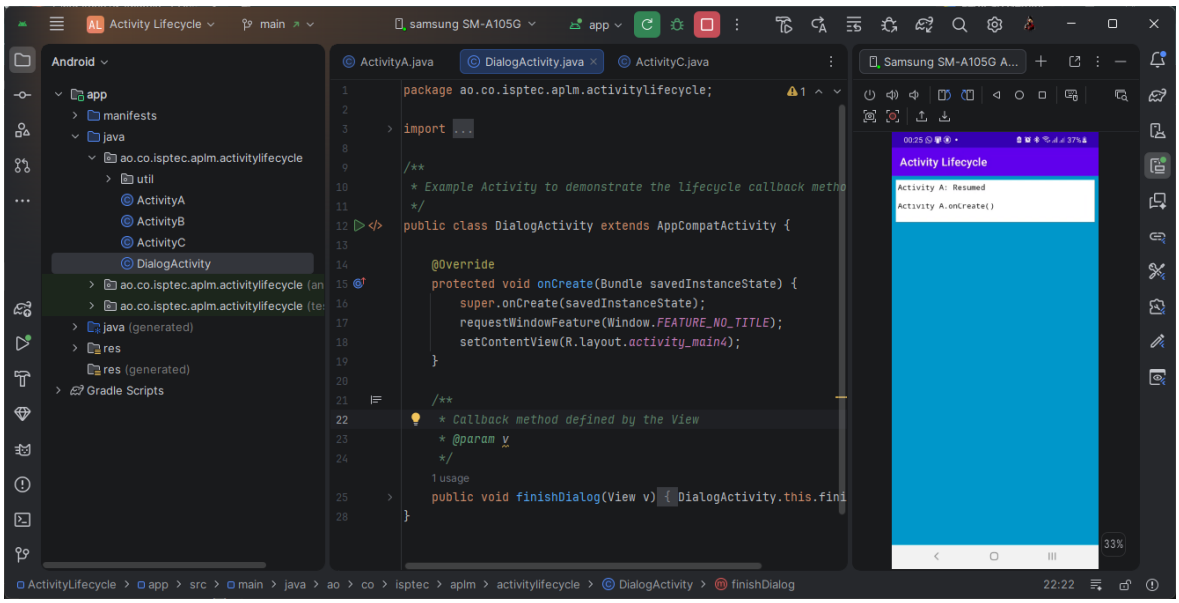


Figura 12 **–** Layout original com raiz `ConstraintLayout` sem restrições: as vistas sobrepõem-se no canto superior do ecrã

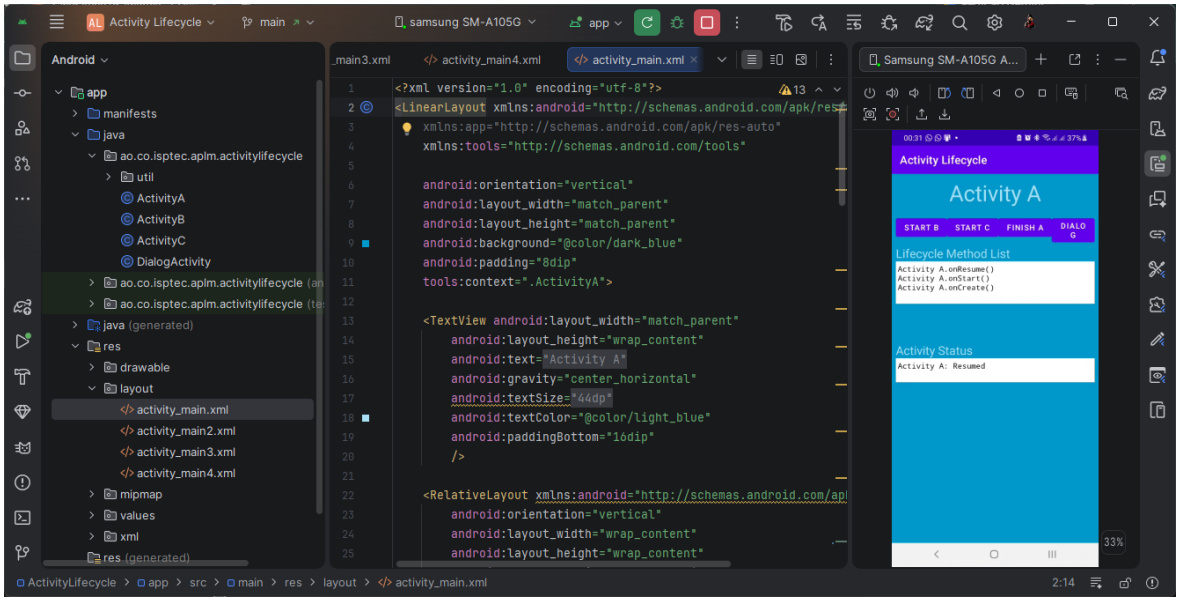
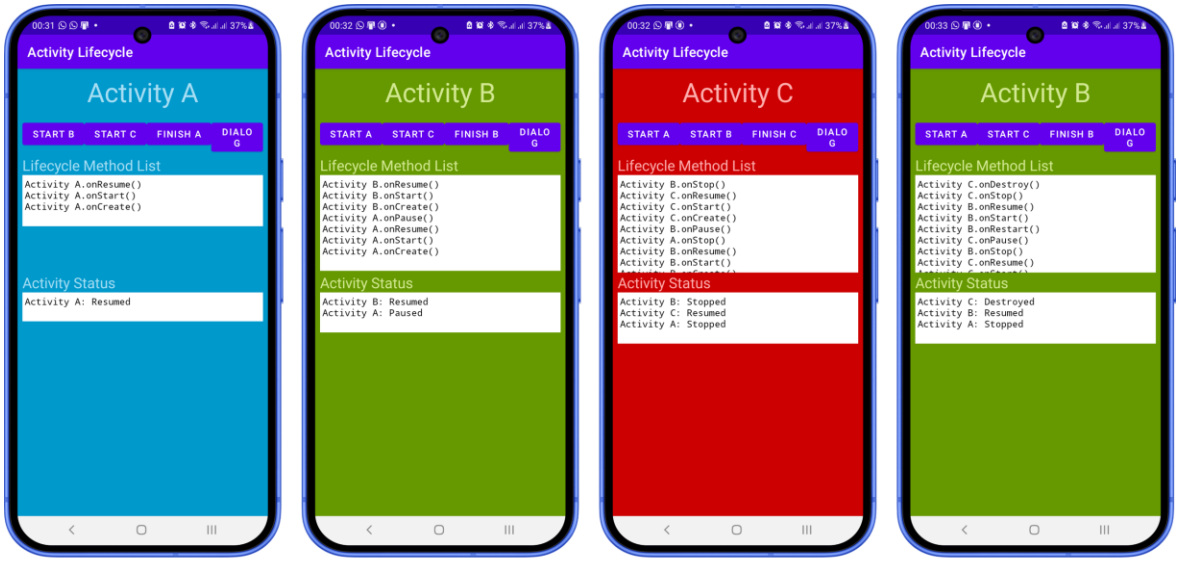


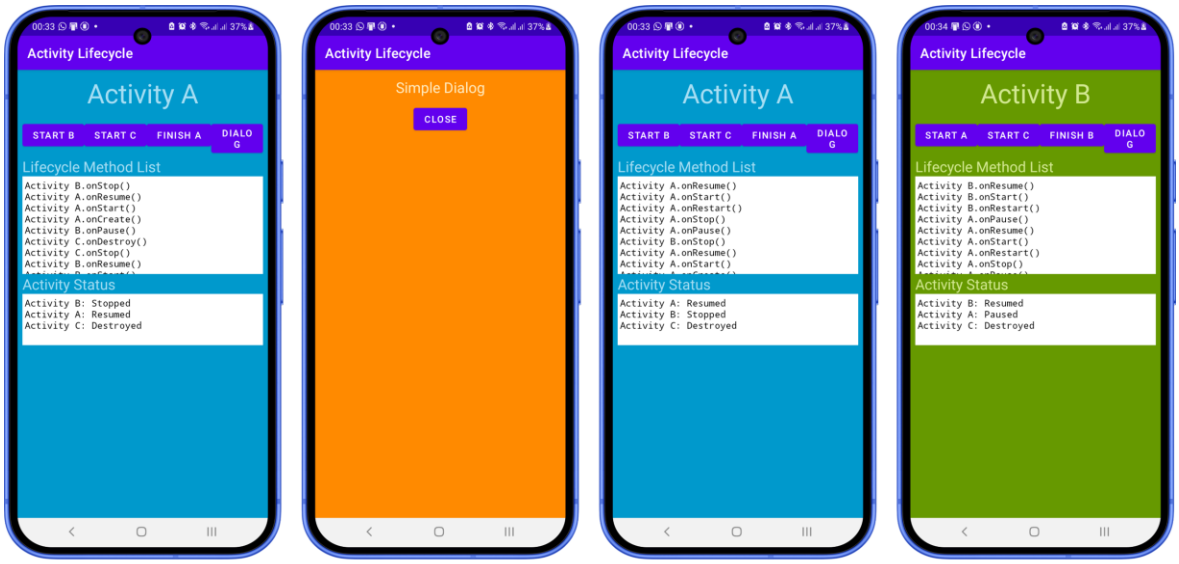
Figura 13 **–** Layout corrigido com raiz `LinearLayout`: título, botões e listas dispostos verticalmente na `ActivityA`

Cada actividade regista os seus callbacks no `StatusTracker` e chama `Utils.printStatus()`, que, com 750 ms de atraso, escreve a lista de métodos (o mais recente no topo) e o estado de cada actividade. A aplicação foi percorrida pela ordem das Figuras 14 e 15.



(a) (b) (c) (d)

Figura 14 **–** Navegação na ActivityLifecycle: (a) arranque da `ActivityA`; (b) «Start B»; (c) «Start C» a partir de B; (d) «Finish C», com regresso à `ActivityB`



(a) (b) (c) (d)

Figura 15 **–** Navegação na ActivityLifecycle: (a) «Start A» a partir de B, que cria uma nova instância de A; (b) `DialogActivity` aberta; (c) após «Close»; (d) botão Voltar, com regresso à `ActivityB`

a) Como se interpreta a saída produzida pela aplicação ao navegar entre as actividades?

R:

A saída confirma a ordem em que o sistema coordena duas actividades: ao lançar B a partir de A, `A.onPause` ocorre primeiro, seguem-se `B.onCreate`, `B.onStart` e `B.onResume`, e só no fim `A.onStop`, quando A deixa de ser visível (Figura 14b). É esta sobreposição que justifica o atraso de 750 ms do `printStatus`. Ainda assim, o `onStop` e o `onDestroy` não chamam `printStatus`: só aparecem se outra actividade imprimir depois deles, razão por que A ainda surge como «Paused» na Figura 14b.

«Finish C» destrói C (`C.onPause`, `C.onStop`, `C.onDestroy`) e retoma B com `onRestart`, porque B estava parada na pilha (Figura 14d). «Start A» a partir de B não regressa à A

original: com o modo de lançamento padrão, cria uma nova instância (`A.onCreate`), ficando a pilha A → B → A (Figura 15a). Como o `StatusTracker` usa o nome como chave, as duas instâncias de A partilham uma só linha no «Activity Status».

A `DialogActivity`, sem tema de diálogo no manifesto, ocupa o ecrã inteiro e opaco: por isso A chega a `onStop` e, ao fechar, retoma com `onRestart` (Figura 15c); com um tema translúcido, A ficaria apenas em `onPause`. O botão Voltar equivale a `finish()` sobre a instância do topo: B é retomada (`onRestart`, `onStart`, `onResume`) e a segunda A é destruída (Figura 15d). Como o `onDestroy` de A chama `mStatusTracker.clear()`, a destruição de qualquer instância de A apaga o histórico partilhado.

b) Qual é o wireframe das actividades da aplicação?

R:

As caixas representam as actividades e as setas as transições desencadeadas pelos botões (Figura 16). `ActivityA` é a actividade de entrada (intent-filter LAUNCHER); cada actividade lança as outras duas e o diálogo, e «Finish X», «Close» e o botão Voltar removem a actividade do topo da pilha.

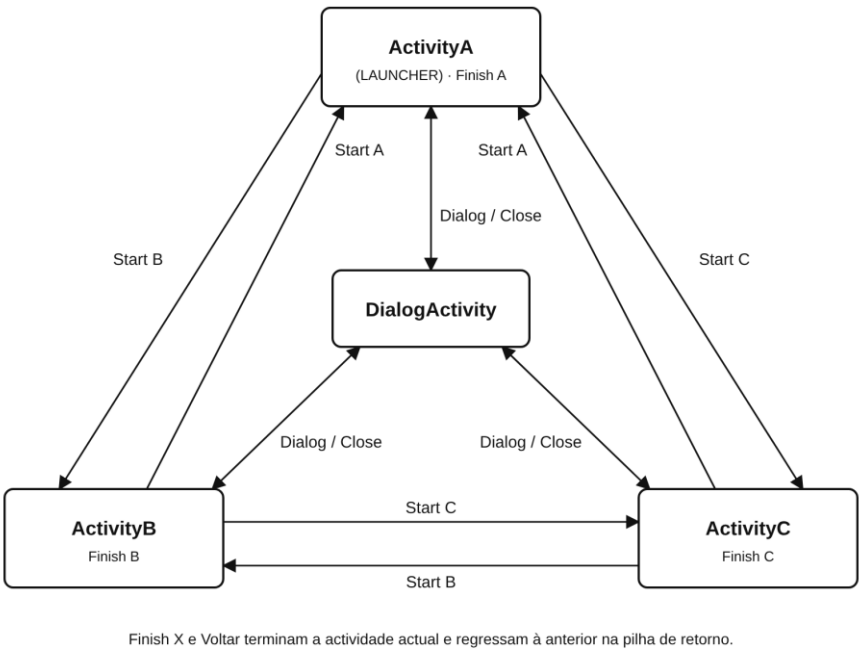


Figura 16 **–** Wireframe das actividades da aplicação ActivityLifecycle e transições entre elas

2.2. Exercício II - Serviços

O objectivo deste exercício é compreender o funcionamento de um serviço iniciado e a partilha de estado entre componentes através do contexto da aplicação.

2.2.1. Ciclo de vida do serviço

O projecto ServiceSimple foi importado da mesma forma, desta vez com o Gradle JDK 17, transferido pelo próprio Android Studio (Figura 17). É composto pela `MainActivity`, com os botões «Start Service» e «Stop Service», e pelo serviço `MyService`, declarado no manifesto com o elemento `<service>`.

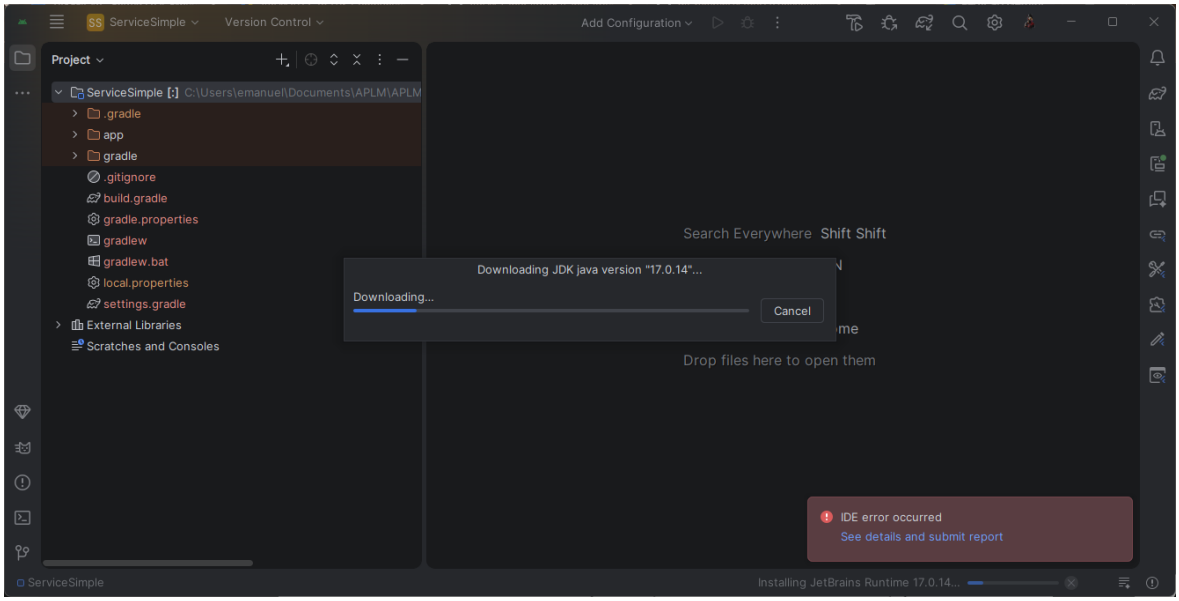


Figura 17 – Importação do projecto ServiceSimple: transferência do JDK 17 para o Gradle

Tal como no projecto anterior, o `activity_main.xml` tinha como raiz um `ConstraintLayout`, embora os botões usem atributos de `RelativeLayout` (`layout_below`, `layout_alignParentTop`, `layout_centerHorizontal`); a raiz foi substituída por `RelativeLayout` (Figura 18). Os botões estão ligados aos métodos da actividade pelo atributo `android:onClick`:

```
public void startService(View view) {
    startService(new Intent(this, MyService.class));
}

public void stopService(View view) {
    stopService(new Intent(this, MyService.class));
}
```

O serviço apresenta um Toast em cada callback do seu ciclo de vida:

```
@Override
public void onCreate() {
    Toast.makeText(this, "Service was Created", Toast.LENGTH_SHORT).show();
}

@Override
public int onStartCommand(Intent intent, int flags, int startId) {
    Toast.makeText(this, "Service Started", Toast.LENGTH_SHORT).show();
    return START_STICKY;
}

@Override
public void onDestroy() {
    Toast.makeText(this, "Service Destroyed", Toast.LENGTH_SHORT).show();
}
```

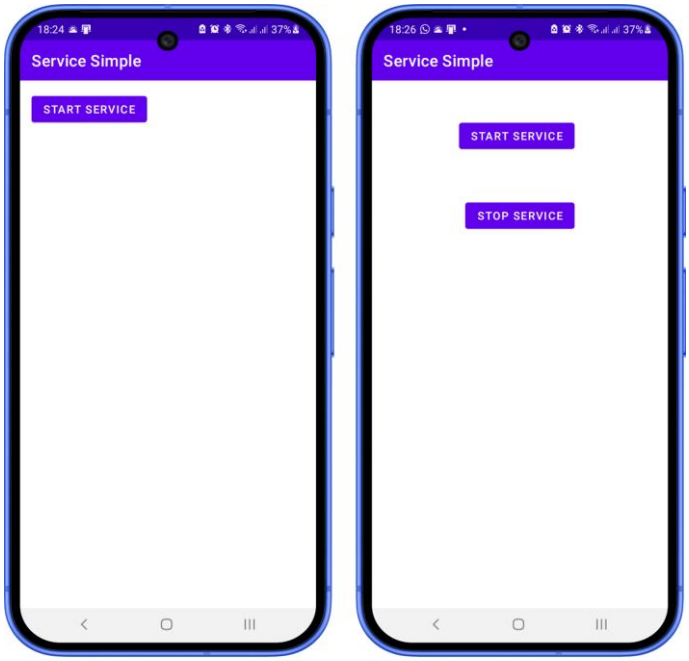


Figura 18 **–** Interface da aplicação ServiceSimple: (a) layout original, com o botão «Stop Service» oculto pela sobreposição; (b) após a substituição da raiz por `RelativeLayout`

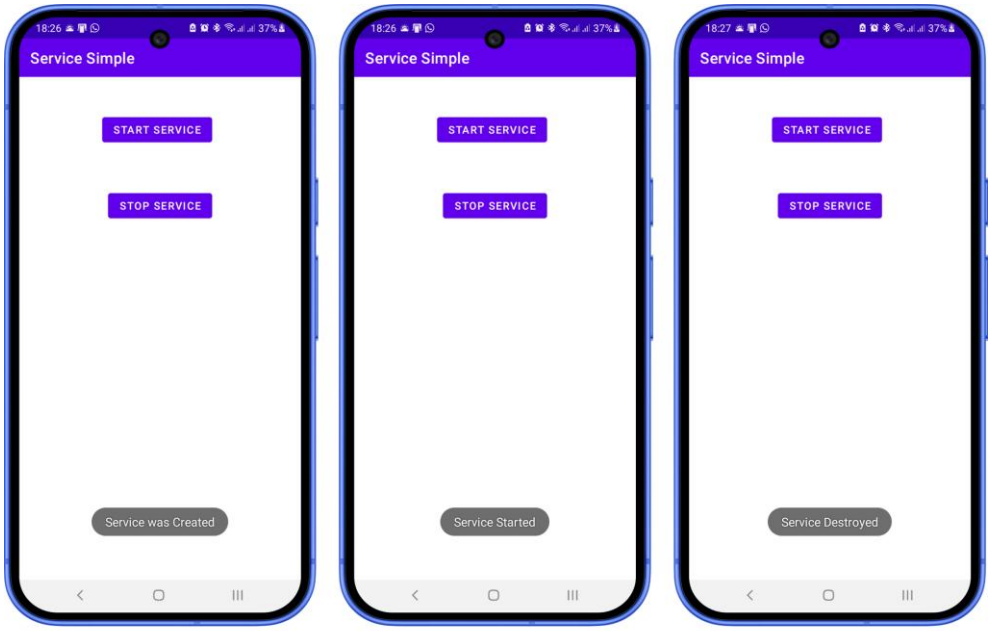


Figura 19 **–** Toasts apresentados pelo serviço: (a) «Service was Created»; (b) «Service Started»; (c) «Service Destroyed»

1) Que sequência de Toasts é apresentada ao pressionar «Start Service» e «Stop Service», e porquê?

R:

«Service was Created» → «Service Started» → «Service Destroyed» (Figura 19). O primeiro `startService()` encontra o serviço inexistente: o sistema cria-o (`onCreate`) e entrega-lhe o pedido (`onStartCommand`). O `stopService()` termina-o e o sistema chama `onDestroy`. Com o serviço já em execução, um novo «Start Service» mostra apenas «Service Started», porque `onCreate` só ocorre uma vez por instância.

O `onBind` nunca é invocado, porque o serviço é iniciado e não vinculado (bound). O valor `START_STICKY` pede ao sistema que recrie o serviço se o processo for terminado por falta de memória. A partir do Android 8 (API 26), um serviço iniciado é parado pelo sistema pouco depois de a aplicação passar a segundo plano, salvo se for promovido a serviço em primeiro plano (foreground service).

2.2.2. Estado partilhado no contexto da aplicação

Para partilhar estado entre a actividade e o serviço, criou-se uma subclasse de `Application`, instanciada uma única vez por processo, antes de qualquer outro componente. Guarda o número de vezes que o serviço foi iniciado:

```
public class ServiceSimpleApp extends Application {
    private int startCount = 0;

    public int getStartCount() {
        return startCount;
    }

    public void incrementStartCount() {
        startCount++;
    }
}
```

A classe é registada no atributo `android:name` do elemento `<application>` do manifesto:

```
<application
    android:name=".ServiceSimpleApp"
    ...>
```

A actividade incrementa o contador imediatamente antes de chamar `startService()`, e o serviço lê-o no `onStartCommand()`. Ambos obtêm o mesmo objecto através de `getApplication()`:

```
// MainActivity
public void startService(View view) {
    ((ServiceSimpleApp) getApplication()).incrementStartCount();
    startService(new Intent(this, MyService.class));
}

// MyService
@Override
public int onStartCommand(Intent intent, int flags, int startId) {
    int count = ((ServiceSimpleApp) getApplication()).getStartCount();
    Toast.makeText(this, "Service Started: " + count, Toast.LENGTH_SHORT).show();
    return START_STICKY;
}
```

Na sequência «Start» → «Stop» → «Start», o contador continua a crescer apesar de o serviço ter sido destruído e recriado (Figura 20), o que confirma que o estado não pertence a nenhum dos dois componentes, mas ao processo. Esse estado sobrevive à recriação da actividade e do serviço, mas perde-se quando o processo termina; para persistir entre execuções, seria necessário guardá-lo em `SharedPreferences` ou numa base de dados.

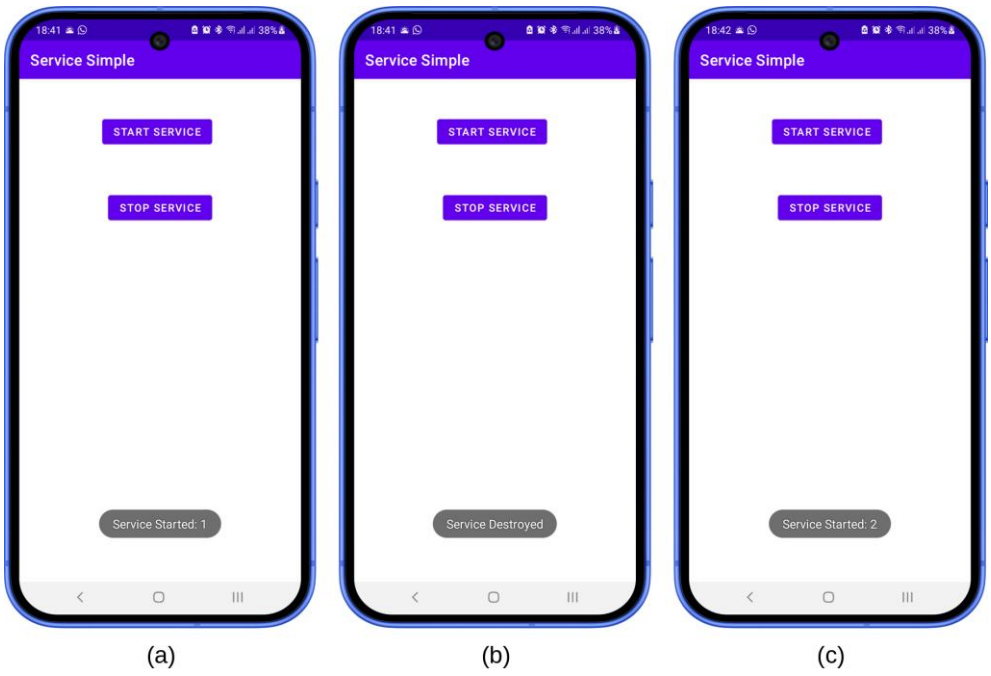


Figura 20 – Contador partilhado no contexto da aplicação: (a) «Service Started: 1»; (b) «Service Destroyed»; (c) «Service Started: 2», após reiniciar o serviço

2.3. Exercício III - Broadcast Receivers

O objectivo deste exercício é definir um broadcast receiver que escute as mudanças de estado do telefone e registe no Logcat cada chamada recebida.

2.3.1. Declaração do receptor

Criou-se o projecto PhoneReceiver (pacote `ao.co.isptec.aplm.phonereceiver`) com uma única actividade, necessária para que o utilizador execute a aplicação e conceda as permissões. O receptor foi criado com **File → New → Other → Broadcast Receiver**, que gera a classe `MyPhoneReceiver` e a regista automaticamente no manifesto. Ao elemento gerado acrescentaram-se o intent-filter para a acção `android.intent.action.PHONE_STATE` e, fora de `<application>`, as duas permissões exigidas:

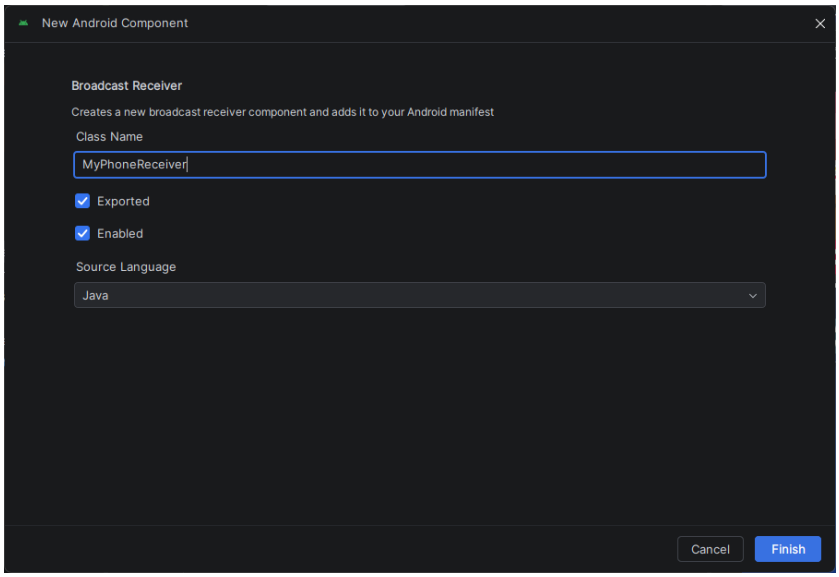


Figura 21 **–** Criação do broadcast receiver `MyPhoneReceiver` com as opções Exported e Enabled activas

```
<uses-permission android:name="android.permission.READ_PHONE_STATE" />
<uses-permission android:name="android.permission.READ_CALL_LOG" />

<application ...>
    <receiver
        android:name=".MyPhoneReceiver"
        android:enabled="true"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.PHONE_STATE" />
        </intent-filter>
    </receiver>
    ...
</application>
```

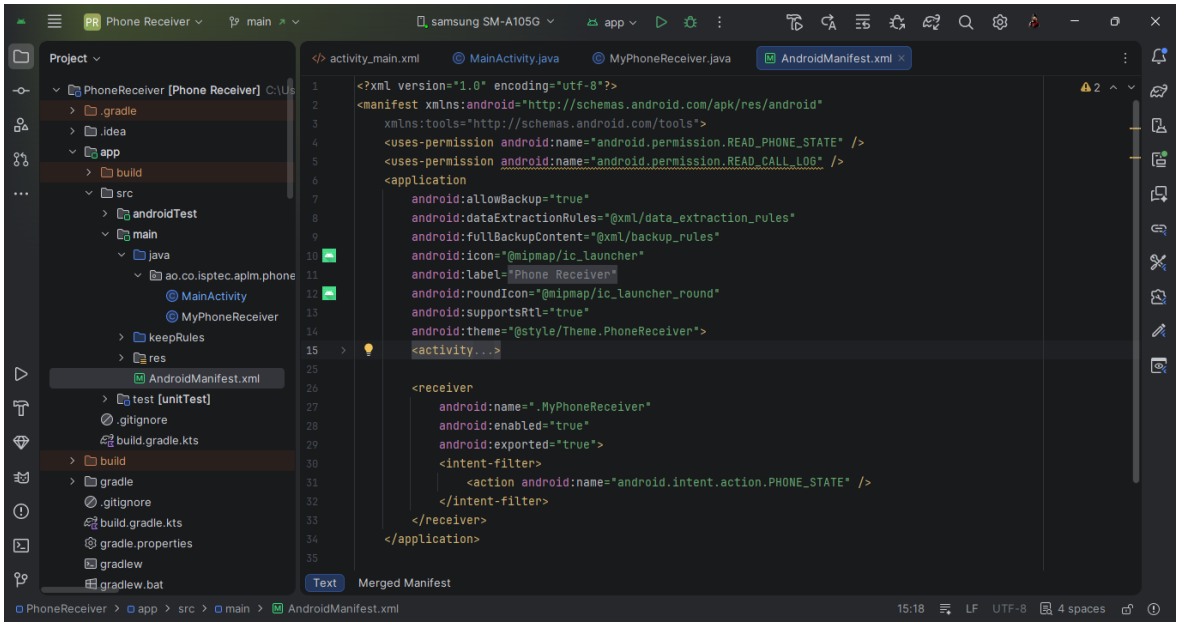


Figura 22 **–** `AndroidManifest.xml` do PhoneReceiver com as permissões e o receptor `MyPhoneReceiver` filtrado pela acção `PHONE_STATE`

No `onReceive()`, o receptor lê dos extras do Intent o estado da chamada e o número de origem e regista-os com `Log.i`. Partiu-se do snippet fornecido (`broadcast_receiver.txt`), substituindo a comparação de strings com `==`, que compara referências e não conteúdos, por `equals()`:

```
public class MyPhoneReceiver extends BroadcastReceiver {

    private static final String TAG = "TAG_MyPhoneReceiver";

    @Override
    public void onReceive(Context context, Intent intent) {
        if (TelephonyManager.ACTION_PHONE_STATE_CHANGED.equals(intent.getAction())) {
            String state = intent.getStringExtra(TelephonyManager.EXTRA_STATE);
            String number = intent.getStringExtra(TelephonyManager.EXTRA_INCOMING_NUMBER);
            Log.i(TAG, "State: " + state + ", Number: " + number);
        }
    }
}
```

1) Como é identificado o novo broadcast receiver no manifesto?

R:

Pelo elemento `<receiver>` dentro de `<application>`, cujo atributo `android:name` aponta para a classe `.MyPhoneReceiver`. O `android:exported="true"` permite receber difusões de fora da aplicação, neste caso do sistema, e o `<intent-filter>` restringe o receptor à acção `PHONE_STATE`.

2.3.2. Permissões em tempo de execução

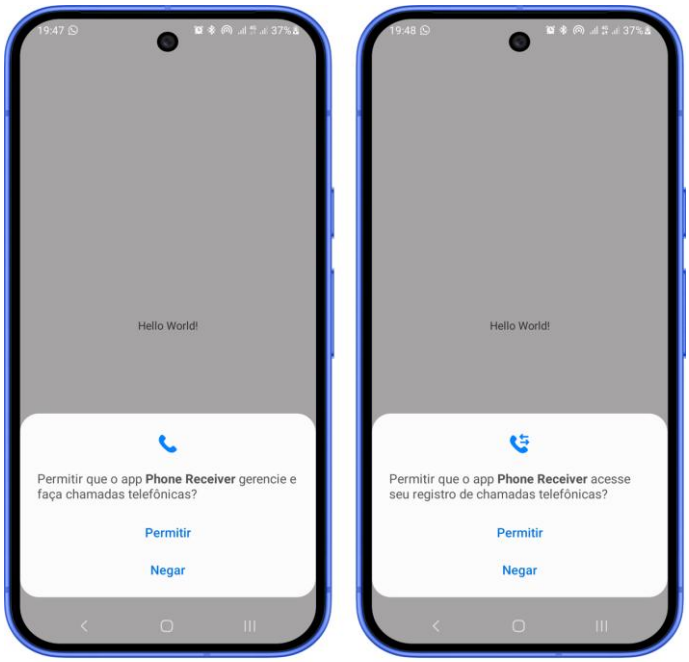
Desde o Android 6.0 (API 23), as permissões perigosas, como `READ_PHONE_STATE` e `READ_CALL_LOG`, têm de ser declaradas no manifesto e também concedidas pelo utilizador durante a execução. O snippet fornecido (`request_explicit_permission.txt`) fazia dois pedidos consecutivos com `requestPermissions()`; como o sistema só apresenta um pedido de cada vez, o segundo é cancelado e o callback regista uma recusa que o utilizador não fez. Agruparam-se, por isso, as permissões em falta num único pedido, feito no `onStart()`:

```
private static final int PHONE_STATUS_REQUEST_CODE = 1;
private static final String[] PHONE_PERMISSIONS = {
        Manifest.permission.READ_PHONE_STATE,
        Manifest.permission.READ_CALL_LOG
};

private void askPhonePermission() {
    ArrayList<String> missing = new ArrayList<>();
    for (String permission : PHONE_PERMISSIONS) {
        if (ContextCompat.checkSelfPermission(this, permission)
                != PackageManager.PERMISSION_GRANTED) {
            missing.add(permission);
        }
    }
    if (!missing.isEmpty()) {
        ActivityCompat.requestPermissions(this, missing.toArray(new String[0]),
                PHONE_STATUS_REQUEST_CODE);
    }
}
```

O resultado chega a `onRequestPermissionsResult()`, que regista, para cada permissão, se foi concedida (Figura 23):

```
@Override
public void onRequestPermissionsResult(int requestCode, String[] permissions, int[]
grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    if (requestCode == PHONE_STATUS_REQUEST_CODE) {
        for (int i = 0; i < permissions.length; i++) {
            boolean granted = grantResults[i] == PackageManager.PERMISSION_GRANTED;
            Log.i(TAG, permissions[i] + (granted ? " concedida" : " nao concedida"));
        }
    }
}
```

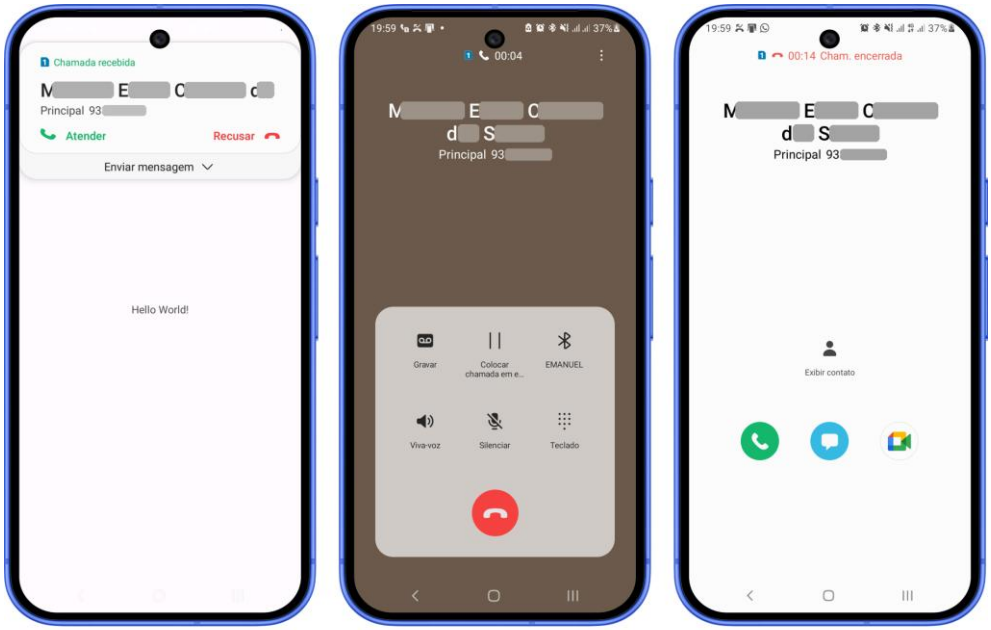


(a) (b)

Figura 23 **–** Pedidos de permissão apresentados pelo sistema ao iniciar a aplicação PhoneReceiver: (a) `READ_PHONE_STATE`; (b) `READ_CALL_LOG`

2.3.3. Teste com uma chamada real

Com a aplicação instalada e as permissões concedidas, o dispositivo recebeu uma chamada real, efectuada a partir de outro telemóvel, que foi atendida e terminada ao fim de alguns segundos (Figura 24). O Logcat, filtrado por `package:mine tag:TAG_MyPhoneReceiver`, confirma que o receptor foi invocado em cada mudança de estado, pela ordem `RINGING` → `OFFHOOK` → `IDLE` (Figura 25). Cada estado surge duas vezes: primeiro com `Number: null` e, alguns milissegundos depois, com o número de origem (no `RINGING`, cerca de dois segundos depois). Num ensaio anterior, com a chamada não atendida, o estado `OFFHOOK` não ocorreu e a sequência foi apenas `RINGING` → `IDLE`. Por se tratar de um contacto real, o nome e o número foram parcialmente ocultados, mantendo-se apenas as iniciais e os dois primeiros dígitos.



(a) (b) (c)

Figura 24 **–** Chamada real recebida no Samsung SM-A105G, com o nome e o número parcialmente ocultados: (a) notificação de chamada recebida; (b) chamada atendida; (c) chamada encerrada

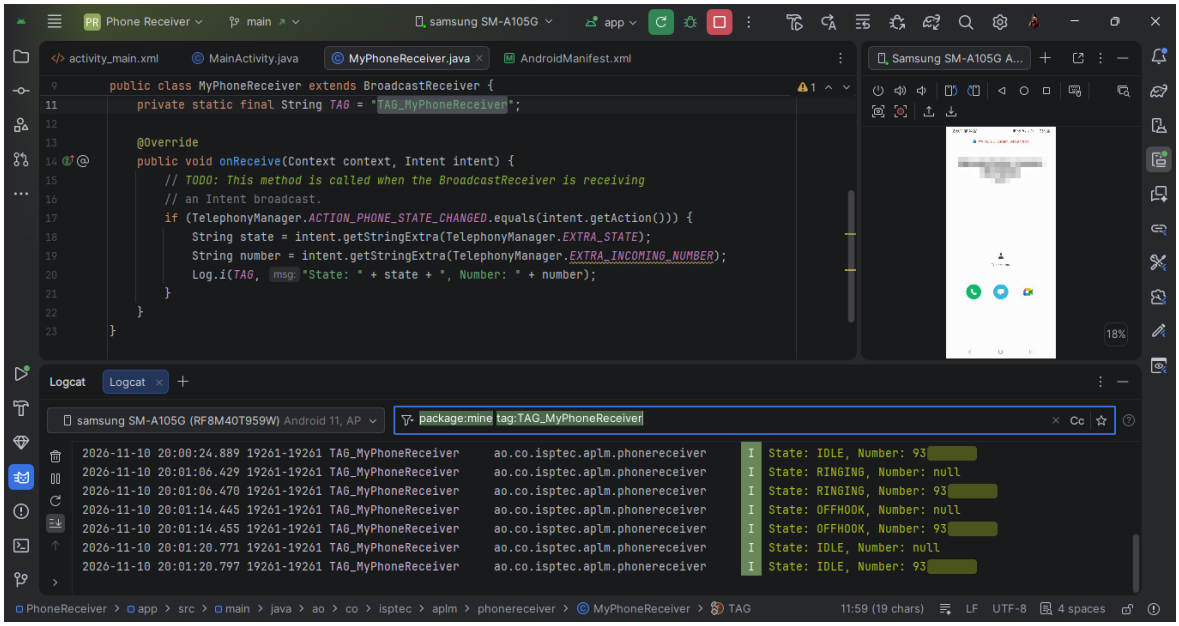


Figura 25 **–** Mensagens do `MyPhoneReceiver` no Logcat durante a chamada: cada estado (`RINGING`, `OFFHOOK`, `IDLE`) é recebido duas vezes, sem e com o número de origem (parcialmente ocultado)

2) Como são preenchidos os extras do Intent `ACTION_PHONE_STATE_CHANGED`?

R:

`EXTRA_STATE` contém sempre o novo estado: `RINGING` (chamada a tocar), `OFFHOOK` (chamada atendida ou efectuada) ou `IDLE` (sem chamada). `EXTRA_INCOMING_NUMBER` só é preenchido para aplicações com `READ_CALL_LOG`. Por isso, uma aplicação que tenha as duas permissões recebe cada difusão duas vezes: uma com o número e outra com o extra em branco (`Number: null`), como se observa na Figura 25; a documentação não garante a ordem entre ambas. O extra `EXTRA_INCOMING_NUMBER` está obsoleto desde a API 29; para identificar chamadas, a documentação recomenda `CallScreeningService`.

3. Desafios

No desafio concebeu-se uma aplicação de bloco de notas com três actividades, seguindo o wireframe do enunciado. A `ListNotesActivity` apresenta os títulos das notas e o botão «New Note»; a `CreateNoteActivity` recolhe o título e o texto de uma nova nota, que é aceite com «OK» ou descartada com «Cancel»; a `ReadNoteActivity` mostra o título e o conteúdo da nota seleccionada. A Tabela 1 resume a estrutura do software.

Tabela 1 **–** Estrutura do software da aplicação Bloco de Notas

Ficheiro	Conteúdo e responsabilidade
`Note.java`	Modelo de dados: título e texto imutáveis; implementa `Serializable`; `toString()` devolve o título
`ListNotesActivity.java`	Actividade de entrada: detém a lista de notas, liga-a ao `ListView` e lança as outras duas actividades
`CreateNoteActivity.java`	Recolhe título e texto e devolve o resultado (`RESULT_OK` ou `RESULT_CANCELED`)
`ReadNoteActivity.java`	Apresenta a nota recebida nos extras do Intent
`activity_list_notes.xml`	`ListView` com peso 1 e botão «New Note» centrado
`activity_create_note.xml`	Dois `EditText` (título e texto multilinha) e os botões «OK» e «Cancel»
`activity_read_note.xml`	`TextView` do título e `TextView` do conteúdo dentro de um `ScrollView`
`AndroidManifest.xml`	Declara as três actividades; `ListNotesActivity` com o intent-filter MAIN/LAUNCHER

3.1. Desenho da interface móvel

O wireframe das actividades (Figura 26) parte do proposto no enunciado e acrescenta-lhe as acções que desencadeiam cada transição. As setas contínuas representam o lançamento de uma actividade e as tracejadas o seu fim, com regresso à anterior na pilha de retorno. Todos os layouts usam um `LinearLayout` vertical, em que o elemento principal (a lista, o texto da nota ou o conteúdo) recebe `layout_height="0dp"` e `layout_weight="1"` para ocupar o espaço restante do ecrã.

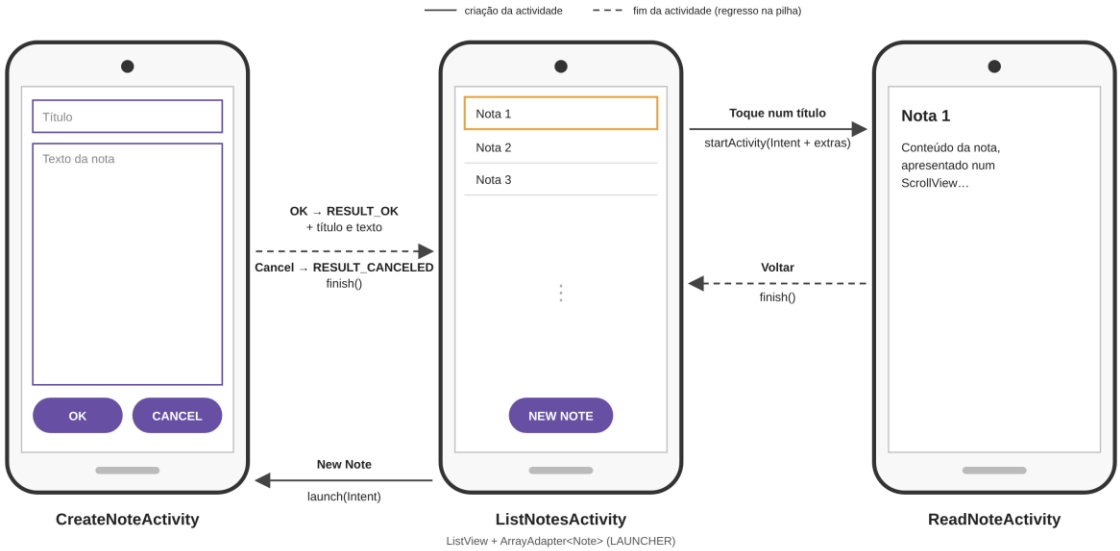
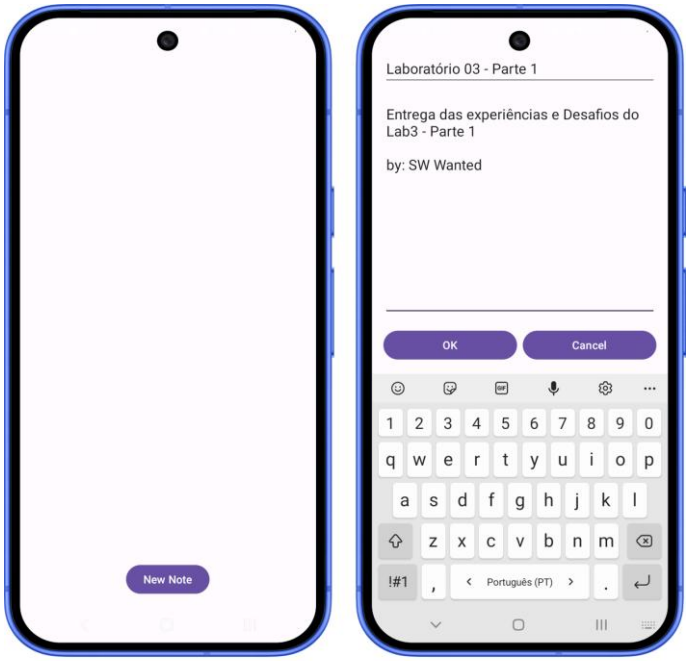


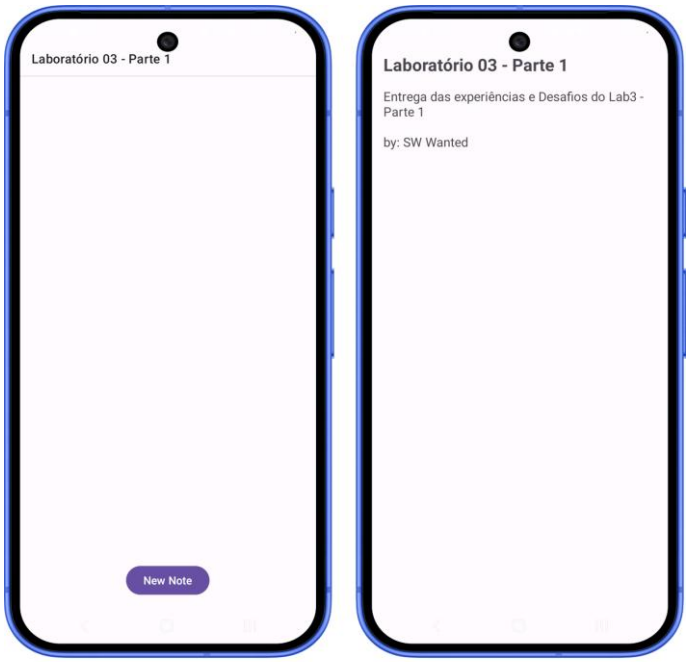
Figura 26 **–** Wireframe das actividades da aplicação Bloco de Notas, com os eventos que desencadeiam cada transição

As Figuras 27 e 28 mostram a aplicação em execução no dispositivo: a criação de uma nota, a sua inserção na lista após «OK» e a respectiva leitura.



(a) (b)

Figura 27 **–** Bloco de Notas no dispositivo: (a) lista vazia com o botão «New Note»; (b) `CreateNoteActivity` com o título e o texto preenchidos



(a) (b)

Figura 28 **–** Bloco de Notas no dispositivo: (a) nota acrescentada à lista após «OK»; (b) `ReadNoteActivity` com o título e o conteúdo da nota seleccionada

3.2. Detalhes de implementação

Arquitectura. A `ListNotesActivity` é a única detentora dos dados; as outras duas actividades não acedem à lista, apenas trocam dados com ela por Intents. A `CreateNoteActivity` devolve a nova nota como resultado e a `ReadNoteActivity` recebe a nota a apresentar nos extras do Intent que a cria. Para obter o resultado usou-se a Activity Result API (`registerForActivityResult` com o contrato `StartActivityForResult`), que substitui o par obsoleto `startActivityForResult`/`onActivityResult`.

As duas actividades secundárias foram criadas com **New → Activity → Empty Views Activity**, que gera a classe, o layout e o registo no manifesto; a opção Launcher Activity ficou desmarcada, para que a `ListNotesActivity` seja a única porta de entrada da aplicação (Figura 29).

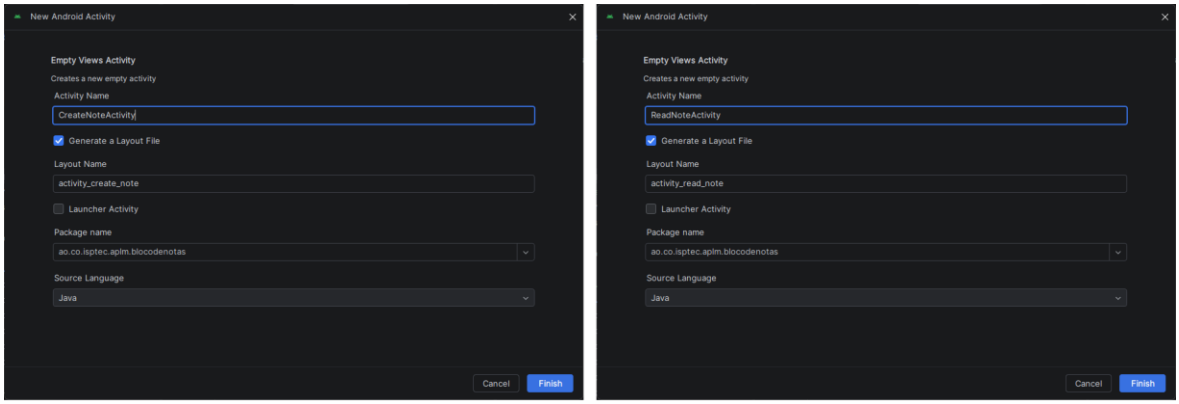


Figura 29 **–** Criação das actividades secundárias, sem a opção Launcher Activity: (a) `CreateNoteActivity`; (b) `ReadNoteActivity`

Estrutura de dados. Cada nota é uma instância da classe `Note`, com dois campos `final`. As notas são guardadas por ordem de criação num `ArrayList<Note>`, ligado ao `ListView` por um `ArrayAdapter<Note>` com o layout de sistema `simple_list_item_1`; como o adapter apresenta o resultado de `toString()`, basta que este devolva o título:

```
public class Note implements Serializable {
    private final String title;
    private final String text;

    // construtor e getters omitidos

    @Override
    public String toString() {
        return title; // texto apresentado pelo ArrayAdapter em cada linha
    }
}

private ArrayList<Note> notes = new ArrayList<>();
private ArrayAdapter<Note> adapter;   // ArrayList -> ListView
```

A inserção no fim do `ArrayList` tem custo O(1) amortizado e o acesso pela posição do clique é O(1). Por implementar `Serializable`, a lista inteira pode ser guardada no `Bundle` em `onSaveInstanceState()` e restaurada no `onCreate()`, o que preserva as notas numa rotação do ecrã.

Eventos. A Tabela 2 resume os eventos tratados pela aplicação.

Tabela 2 **–** Eventos tratados pela aplicação Bloco de Notas

Evento	Componente · callback	Tratamento	Efeito no estado e na UI
Clique em «New Note»	`Button` · `OnClickListener`	`createNoteLauncher.launch(Intent)`	`CreateNoteActivity` em primeiro plano
Clique em «OK»	`Button` · `OnClickListener`	`setResult(RESULT_OK, extras)` e `finish()`	Nota acrescentada; `notifyDataSetChanged()` actualiza a lista
Clique em «Cancel»	`Button` · `OnClickListener`	`setResult(RESULT_CANCELED)` e `finish()`	Nota descartada; lista inalterada
Toque num título	`ListView` · `OnItemClickListener`	`startActivity(Intent + extras)`	`ReadNoteActivity` mostra a nota
Botão Voltar	Sistema · `finish()`	Remove a `ReadNoteActivity` da pilha	Regresso à lista
Rotação do ecrã	`onSaveInstanceState` · `onCreate`	`putSerializable` / `getSerializable`	Lista preservada após a recriação

O resultado da criação é tratado no callback registado na `ListNotesActivity`. A nota só é acrescentada se o código for `RESULT_OK` e o Intent de dados existir; um «Cancel» ou o botão Voltar devolvem `RESULT_CANCELED` e a lista mantém-se:

```
private final ActivityResultLauncher<Intent> createNoteLauncher =
        registerForActivityResult(new ActivityResultContracts.StartActivityForResult(),
result -> {
            Intent data = result.getData();
            if (result.getResultCode() == RESULT_OK && data != null) {
                notes.add(new Note(
                        data.getStringExtra(CreateNoteActivity.EXTRA_TITLE),
                        data.getStringExtra(CreateNoteActivity.EXTRA_TEXT)));
                adapter.notifyDataSetChanged();
            }
        });
```

Na `CreateNoteActivity`, «OK» normaliza o título com `trim()`, atribui «Sem título» a uma nota sem título e devolve os dois campos como extras:

```
private void acceptNote() {
    String title = editTitle.getText().toString().trim();
    if (title.isEmpty()) {
        title = "Sem título";
    }
    Intent result = new Intent();
    result.putExtra(EXTRA_TITLE, title);
    result.putExtra(EXTRA_TEXT, editText.getText().toString());
    setResult(RESULT_OK, result);
    finish();
}
```

Comportamento. A Figura 30 mostra o tratamento de cada evento e o efeito resultante na interface.

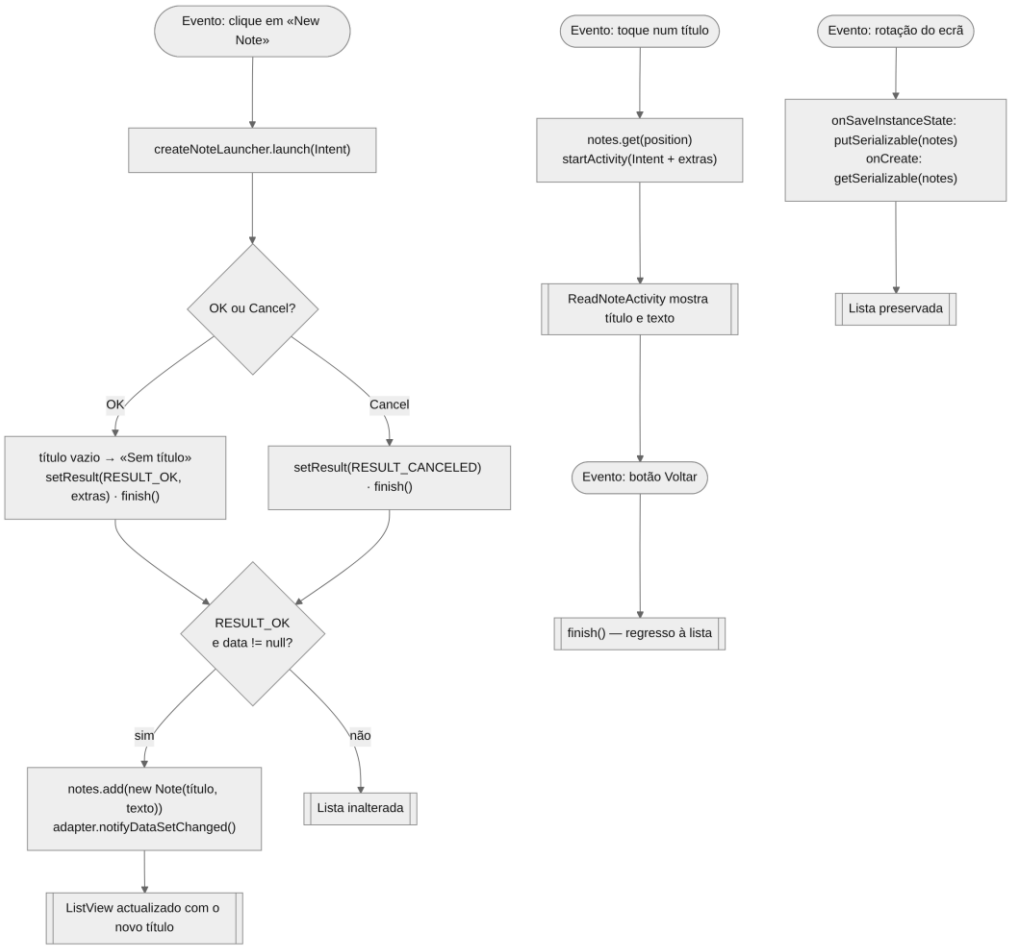


Figura 30 **–** Comportamento da aplicação Bloco de Notas: criação, aceitação ou cancelamento de uma nota, leitura e preservação da lista numa rotação

Outros detalhes técnicos. Na `CreateNoteActivity`, os insets do teclado (`WindowInsetsCompat.Type.ime()`) somam-se aos das barras do sistema, para que os botões «OK» e «Cancel» permaneçam visíveis com o teclado aberto. Como limitação, as notas existem apenas em memória: sobrevivem a uma rotação, mas perdem-se quando a aplicação é fechada ou o processo terminado. A persistência exigiria uma base de dados local (Room) ou, para partilhar as notas com outras aplicações, um content provider.

4. Referências Bibliográficas

Android Developers. (s.d.). *Broadcasts overview*. https://developer.android.com/develop/background-work/background-tasks/broadcasts

Android Developers. (s.d.). *BroadcastReceiver*. Android API reference. https://developer.android.com/reference/android/content/BroadcastReceiver

Android Developers. (s.d.). *Content provider basics*. https://developer.android.com/guide/topics/providers/content-provider-basics

Android Developers. (s.d.). *Get a result from an activity*. https://developer.android.com/training/basics/intents/result

Android Developers. (s.d.). *Introduction to activities*. https://developer.android.com/guide/components/activities/intro-activities

Android Developers. (s.d.). *Request runtime permissions*. https://developer.android.com/training/permissions/requesting

Android Developers. (s.d.). *Run apps on the Android Emulator*. https://developer.android.com/studio/run/emulator

Android Developers. (s.d.). *Services overview*. https://developer.android.com/develop/background-work/services

Android Developers. (s.d.). *Tasks and the back stack*. https://developer.android.com/guide/components/activities/tasks-and-back-stack

Android Developers. (s.d.). *TelephonyManager*. Android API reference. https://developer.android.com/reference/android/telephony/TelephonyManager

Android Developers. (s.d.). *The activity lifecycle*. https://developer.android.com/guide/components/activities/activity-lifecycle

Android Developers. (s.d.). *View logs with Logcat*. https://developer.android.com/studio/debug/logcat

Costa, J. (2026). *Laboratório #03 – Parte 1: Componentes da aplicação Android* [Enunciado de laboratório, Aplicações Móveis, ano lectivo 2026-27]. Instituto Superior Politécnico de Tecnologias e Ciências (ISPTEC).

Rittmeyer, W. (s.d.). *Android tutorial: BroadcastReceiver*. Grokking Android. http://www.grokkingandroid.com/android-tutorial-broadcastreceiver/

Steele, J., & To, N. (2010). *The Android developer's cookbook: Building applications with the Android SDK*. Addison-Wesley

5. Repositório GitHub

Repositório: github.com/SW-Wanted/APLM-Laboratorios.[1]

[1] Foi enviado um convite de colaborador para o utilizador **joaojdacosta**.